Db2 for i

Un héritage puissant qui ne demande qu'à évoluer

Andy — Ingénieur de données

SINAPSE

iBelieve 2026 · 15 octobre 2026

iBelieve 2026 Présent et Futur de l'IBM i

Co-organisé par itheis IBM

Qui suis-je ?

Andy

Ingénieur de données · Sinapse

- iBelieve 2021 : **500+ tables DDS → SQL**
- **Fan assumé de l'IBM i**, depuis mes années DBA
- **Le passeur** : l'existant IBM i ↔ la data moderne

avant 2016
Bureautique
déjà du SQL

→

2016 – 2019
Développeur BI
Cognos
consomme la base

→

2019 – 2022
DBA / DBE Db2 for i
IBM i SQL
500+ tables → SQL

→

2022 – 2026
Architecte données
CAFAT
dbt · dlt · Kestra · ClickHouse

→ continuité

2026 →
Ingénieur données
SINAPSE

Sinapse : la suite de la CAFAT

GIE · 2024
CAFAT · Gouvernement · FSH

265 000
Calédoniens concernés

Étape par étape
le recouvrement d'abord

Membres fondateurs

CAFAT — Votre vie, c'est notre quotidien
Gouvernement de la Nouvelle-Calédonie
fonds social de l'habitat — Bien plus qu'un toit

→ SINAPSE → étape par étape → **SI social modernisé**

Hier **architecte de données à la CAFAT**, aujourd'hui je **construis la suite**.

sinapse.nc — Qui sommes-nous ? · sinapse.nc — Projet · sinapse.nc

Faire évoluer son IBM i : par où commencer ?

40 ans d'applicatif, des règles dans les programmes, peu de documentation



Avec un outil

jusqu'où aident-ils ?



Sans outil

comment avancer quand même ?

IBM i

Avec le natif seul

que permet le SQL ?

Le fil de la réponse : le SQL, **bien plus que des transactions**



moderniser



administrer



exploiter



ouvrir les données

1. Le chantier 2019-2022

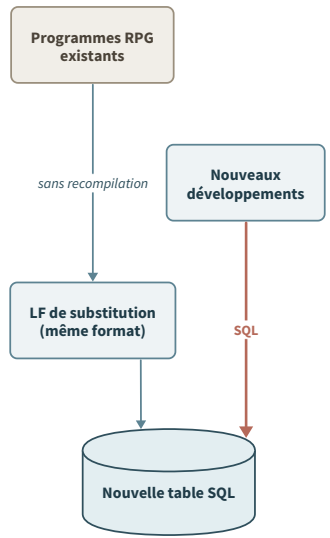
500+

tables DDS → SQL, **sans recompiler**

itheis GAIA resolution XCASE

ET APRÈS

contraintes **temporel** **RCAC**



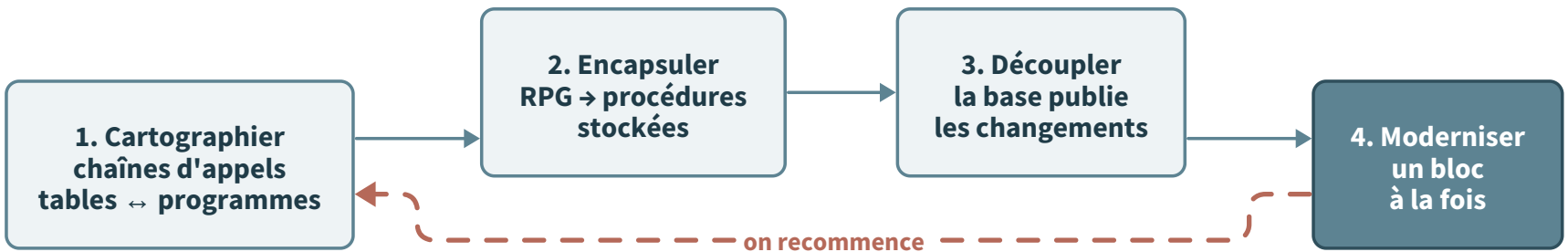
❤ Merci à **E. Sadiman**, qui m'a confié ce chantier — et m'a rendu fan de l'IBM i

> iBelieve 2021 : **migrer des tables.** iBelieve 2026 : en faire **une plateforme de données.**

Source : Itheis — méthode surrogate

2. Moderniser progressivement

On ne réécrit pas 40 ans d'applicatif d'un coup

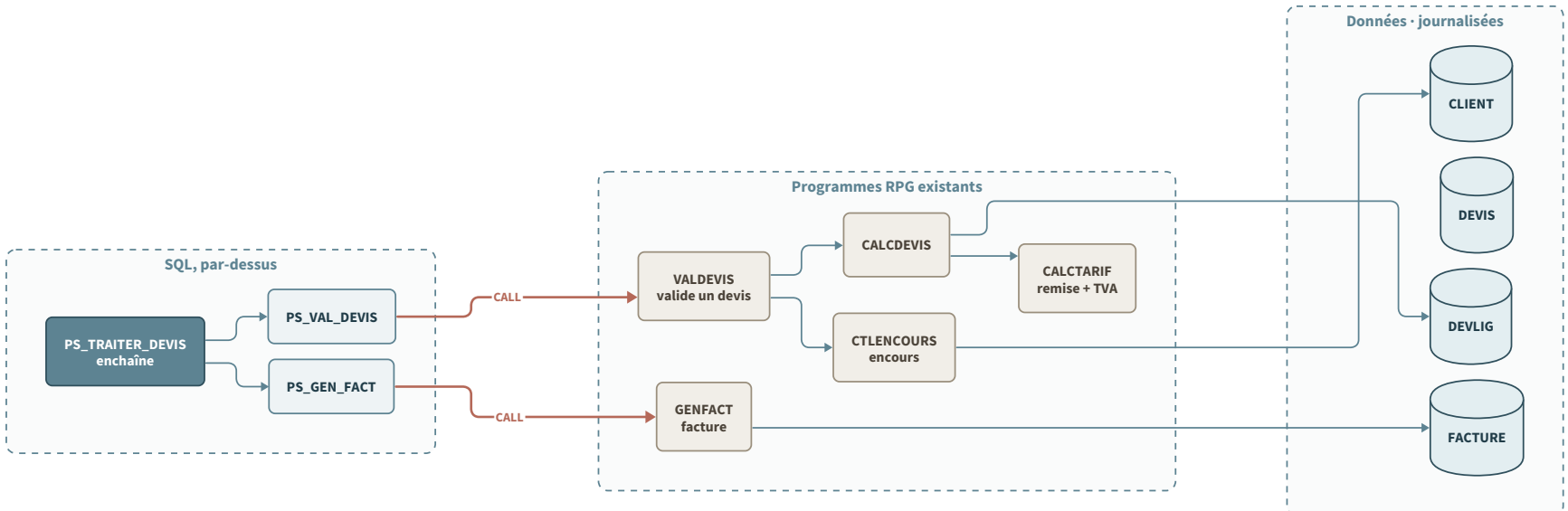


Le SQL est l'outil de chacune de ces étapes.

Principes : SG24-8185 · Redbook 2026 MD260020 · REDP-5095

Le terrain : une mini-application de devis

✓ Tout le SQL qui suit a **tourné** sur pub400 — et se rejoue avec le **kit** du dépôt



Kit : `kit/rpg/` — 5 tables, 7 programmes RPG compilés depuis ACS, 3 procédures SQL · sans compilateur : `kit/00-terrain.sql`

2.1 Cartographier

En clair : savoir qui appelle quoi avant de toucher à quoi que ce soit.



Avec un outil

X-Analysis : modèle déduit du code



Avec l'IA

Bob ou un autre assistant lit le RPG

IBM i

Sans outil

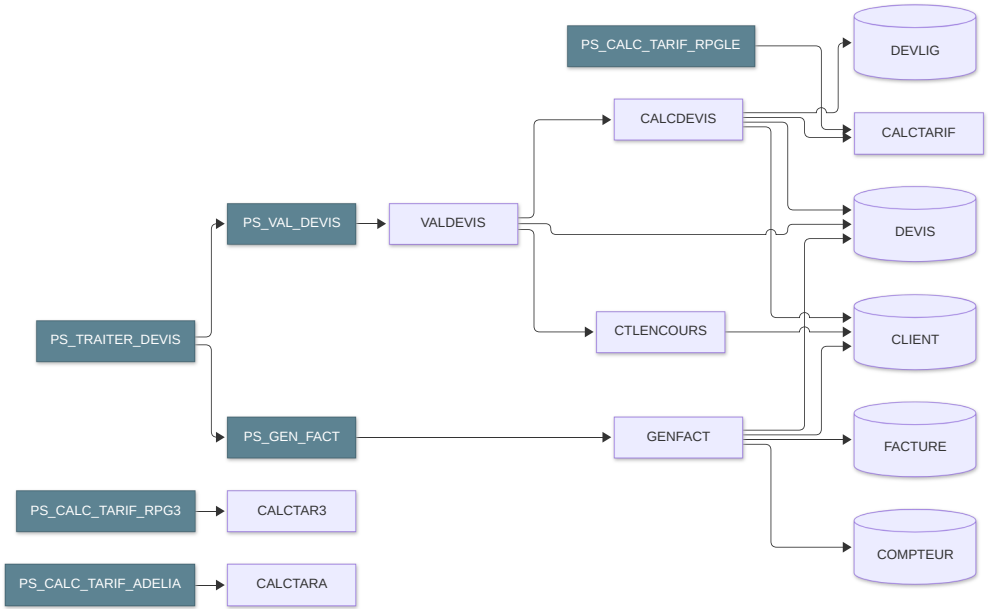
DSPPGMREF, piloté en SQL

```
CALL QSYS2.QCMDEXC(
  'DSPPGMREF PGM(' CONCAT CURRENT SCHEMA CONCAT '/*ALL) OUTPUT(*OUTFILE) OBJTYPE(*PGM) '
  CONCAT 'OUTFILE(' CONCAT CURRENT SCHEMA CONCAT '/T_PGMREF) OUTMBR(*FIRST *REPLACE)');
```

8 programmes analysés → **3** appels entre eux + **9** références de tables

X-Analysis · DSPPGMREF · Format QADSPPGM · IBM Bob pour IBM i · IT Jungle — Bob 1.0 · `kit/rpg/15-cartographier-rpg.sql`

La chaîne d'appels, du SQL jusqu'au RPG



Le même graphe, écrit en Mermaid par une requête SQL (kit).

> Le natif ne remplace pas un outil. Il permet de savoir **par où commencer** et **quoi ne pas casser**.

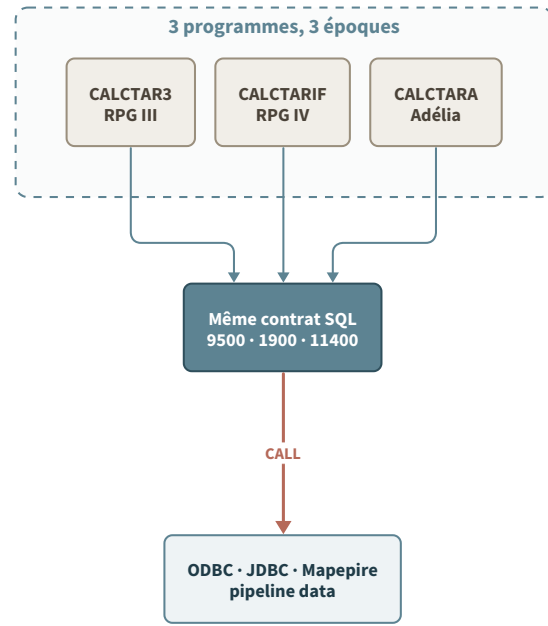
SYSROUTINEDEP · SYSROUTINES · Bob Cozzi — DSPPGMREF en UDTF · `kit/rpg/15-cartographier-rpg.sql`

2.2 Encapsuler

Un programme existant devient une **procédure stockée**. Sans toucher à son code.

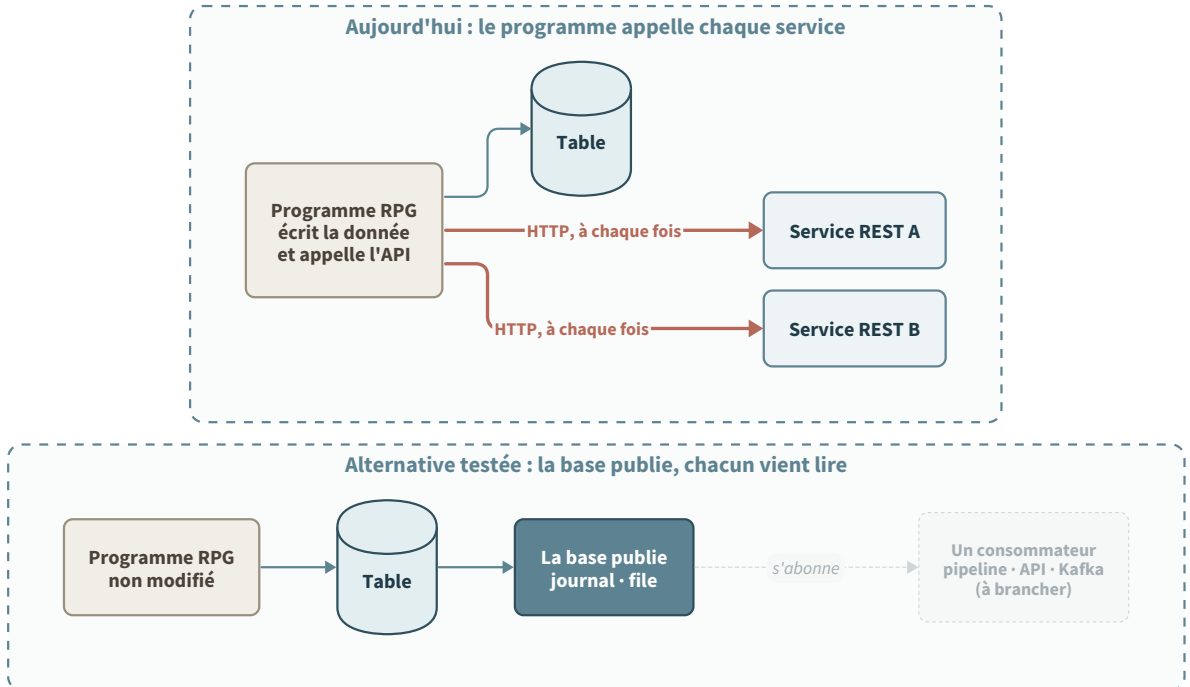
```
CREATE OR REPLACE PROCEDURE PS_CALC_TARIF_RPG3 (
    IN  p_montant_ht   DECIMAL(9,2),
    IN  p_taux_remise  DECIMAL(5,2),
    OUT p_remise       DECIMAL(11,2),
    OUT p_tva          DECIMAL(11,2),
    OUT p_ttc          DECIMAL(11,2),
    OUT p_rc           CHAR(1)
)
  LANGUAGE RPG                      -- RPG, pas RPGLE : c'est de l'OPM
  SPECIFIC PS_TARIF3
  NO SQL
  EXTERNAL NAME 'VOTREBIB/CALCTAR3'
  PARAMETER STYLE GENERAL;

CALL PS_CALC_TARIF_RPG3(10000.00, 5.00, ?, ?, ?, ?);
```



2.3 Découpler : publier sans toucher aux programmes

En clair : au lieu que le programme appelle chaque service, la base publie et chacun vient lire.



Le programme n'a pas changé, et **la base publie** ce qui change.

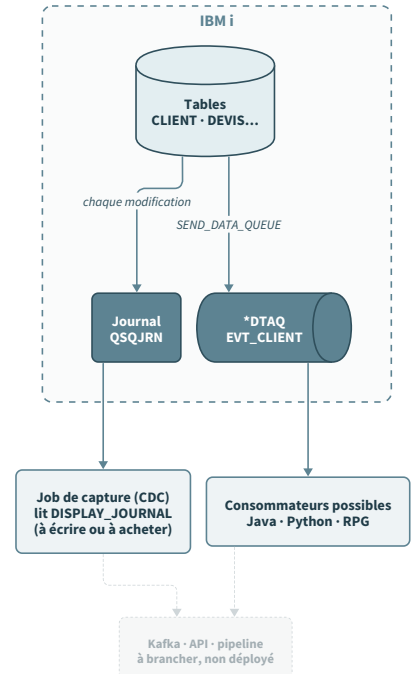
Testé sur pub400 : journal et file. Aucun consommateur n'est déployé.

2.3 Le journal et la file : la base publie

Le consommateur lit, y compris par ODBC

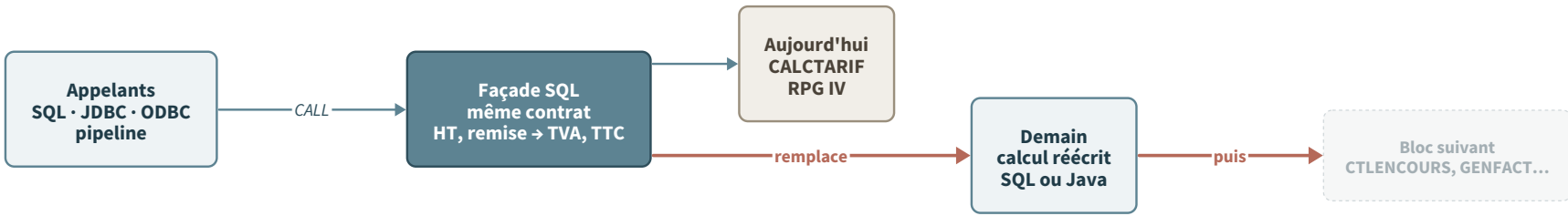
```
-- Côté consommateur, y compris via ODBC
SELECT CAST(MESSAGE_DATA_UTF8 AS VARCHAR(1000)) AS MESSAGE
  FROM TABLE (QSYS2.RECEIVE_DATA_QUEUE(
         DATA_QUEUE         => 'EVT_CLIENT',
         DATA_QUEUE_LIBRARY => CURRENT SCHEMA));
```

```
{"table":"CLIENT","id":1001,"encours":2910.00,
 "ts":"2026-09-29-09.45.57.398581"}
```



2.4 Moderniser, un bloc à la fois

En clair : on remplace ce qu'il y a derrière la façade, les appelants ne voient rien.



Le contrat ne change pas, **l'implémentation si**.

Principe, non exécuté : le kit montre trois implémentations derrière un même contrat (2.2), pas encore une réécriture.

3. Administrer et exploiter en SQL

En clair : le système se consulte comme une table, sans connaître les commandes.

Une commande CL, un service SQL

Commande CL	Service SQL
`WRKACTJOB`	`QSYS2.ACTIVE_JOB_INFO`
`DSPLOG`	`QSYS2.HISTORY_LOG_INFO`
`DSPJRN`	`QSYS2.DISPLAY_JOURNAL`
`WRKOBJOWN`	`QSYS2.OBJECT_OWNERSHIP`
`WRKSYSVAL`	`QSYS2.SYSTEM_VALUE_INFO`

Les exemples sont livrés



Capture à venir

ACS → Run SQL Scripts → Insert from Examples

`captures/acs-insert-from-examples.png · capture fixe`

Sécurité · exploitation · audit

Tout peut partir vers un SIEM, par le chemin de 2.3



La sécurité de l'IBM i rejoint **la supervision de toute l'entreprise**.

Principe, non exécuté : aucun SIEM branché sur pub400.

SYSTEM_VALUE_INFO · HISTORY_LOG_INFO · OBJECT_PRIVILEGES · Syslog · Graylog · ELK · ClickStack

4. L'IBM i, source de données de premier plan

4.1 Se connecter

En clair : les données de l'IBM i alimentent les outils modernes, sans recopie à la main.



Les procédures de l'étape 2.2 passent **par les mêmes portes**

ODBC ACS · ODBC Linux · JTOpen · Mapepire · MCP

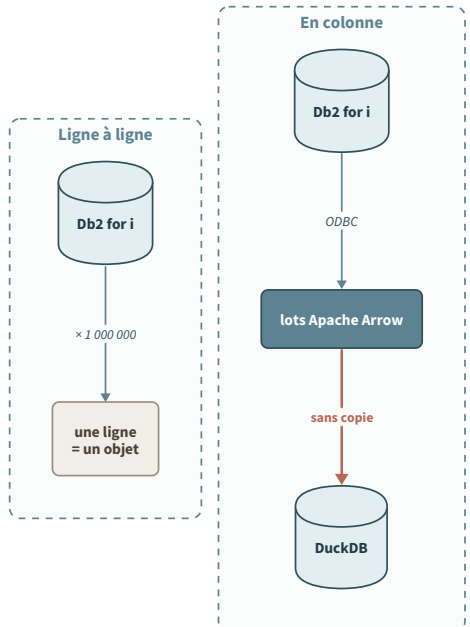
4.2 ODBC → DuckDB, avec ou sans Arrow

✓ Exécuté sur pub400 le 30/09, **en connexion directe** : le pilote IBM i Access ODBC et un DSN.

Pour le volume : des lots Arrow, en colonne

```
reader = read_arrow_batches_from_odbc(
    query="SELECT * FROM VOTREBIB.CLIENT",
    connection_string="DSN=MONIBMI",
    batch_size=50_000,
)
clients = pa.Table.from_batches(
    reader, schema=reader.schema)

duckdb.sql(
  "CREATE TABLE client AS SELECT * FROM clients")
```



Dans les deux cas, dans DuckDB :

clients	encours_total
3	48000.00

1 ligne(s)

DuckDB — extension ODBC · arrow-odbc-py · arrow-odbc (Rust) · `kit/exemples/duckdb-odbc.sql` · `arrow_odbc_duckdb.py`

4.3 Industrialiser l'extraction

dlt + pyarrow

vers Parquet : 20 à 30× plus rapide

Orchestrer

planifier · rejouer · historiser

GARDE-FOUS

- **lecture seule**
- **TLS**
- **typage**
- **hors pics**

Paralléliser par RRN

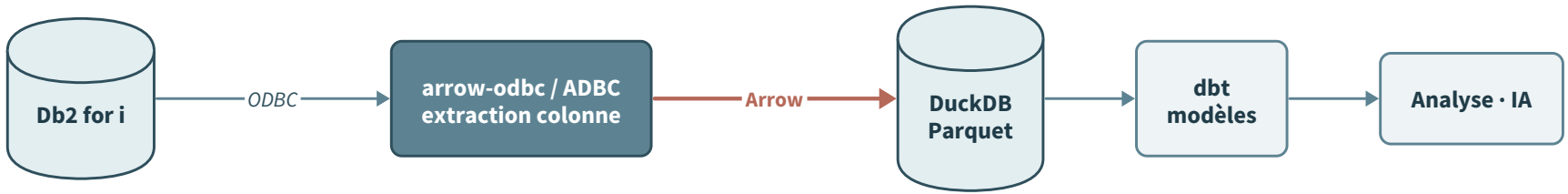
```sql
-- Taille connue du catalogue
SELECT NUMBER_ROWS
  FROM QSYS2.SYSPARTITIONSTAT
 WHERE TABLE_SCHEMA = CURRENT SCHEMA
   AND TABLE_NAME   = 'DEVLIG';

-- Une tranche parmi N
SELECT * FROM DEVLIG D
 WHERE RRN(D) BETWEEN 1 AND 2;
```

Sur le kit : **4** lignes au catalogue → 2 tranches de 2.

dlt sql_database · ODBC Linux + TLS · Query Supervisor

4.4 En vrai : Sinapse construit l'outil qui manque



Le besoin

alimenter la plateforme data, sans peser sur la prod

L'existant

ODBC, fiable mais ligne à ligne

Construit

Arrow · ADBC · piste Flight SQL

Résultat

[volumes, temps avant / après]

Apache Arrow ADBC

SINAPSE

À retenir



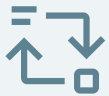
Pas à pas

cartographier · encapsuler · découpler



Le SQL intègre

administre · exploite · publie



Une vraie source

ODBC · Arrow · DuckDB



Un tremplin

l'existant construit la suite

↗ Un moteur qui **évolue** : **17 nouveaux services** à l'été 2026

L’héritage est là.

La seule question, c’est ce qu’on en fait.

Merci — questions ?



SG24-8185 · MD260020 · REDP-5095 · Gists Scott Forstie · Db2 for i Technology Updates
ODBC Linux · arrow-odbc-py · Apache Arrow ADBC · Mapepire · DuckDB ODBC